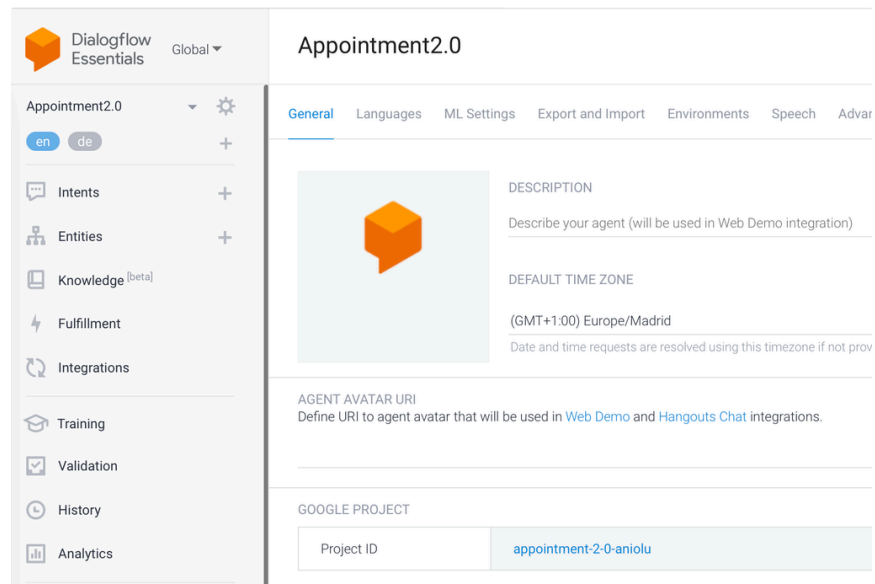


DialogFlow (WIP)

This documents will try to summarize some of the important points that we should know about DialogFlow when working with the Bots.

We will focus on Appointment 2.0 ( Dialogflow), one of the Call Flows that are referring to this bot is https://butterfly.virtualq.io/call_flows/23 (lineup + appointment + phonebot (1) Line Up Or Appointment2).

The way the Bot is connected to the specific Agent on DialogFlow is by the `agent_id` on each `bot`. For instance the Appointment 2 English bot its `agent_id` is `appointment-2-0-aniolu` which is the Project ID that you can get from the settings of that agent (or in the URL):



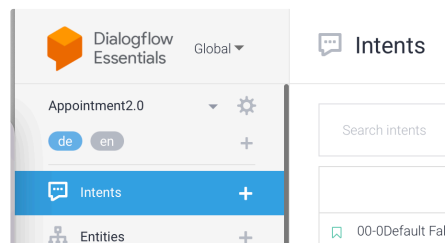
Example of a User interaction

The example we are going to follow is:

1. The User calls the number
2. The bot offers them the next possible appointment
3. The user rejects the offer
4. The bot gives to the user the days and times when is possible to book an appointment (for instance: from Monday to Friday, from 8 AM to 8:50 PM) and asks for which day would like to book the appointment
5. The User gives the answer (eg: Tomorrow)
6. The bot comes back saying that for that day they have free appointments between X and Y, so the user must indicate what is the best option for them
7. User gives a time
8. The bot checks if is possible or not, in case of yes, the user must confirm the appointment, otherwise has the option to go over the cycle again and give a new Time

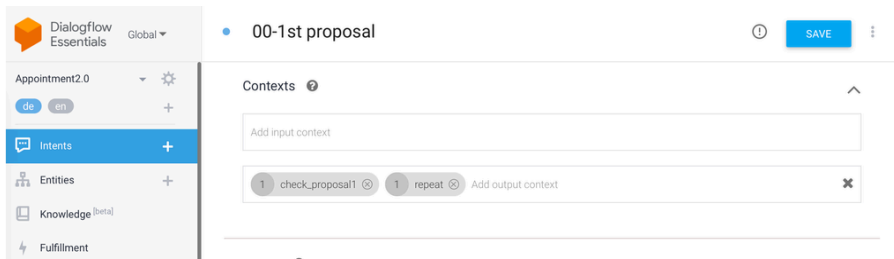
Intents

First thing to notice, is that for the same bot we have different languages (and of course, different `agent_id`):



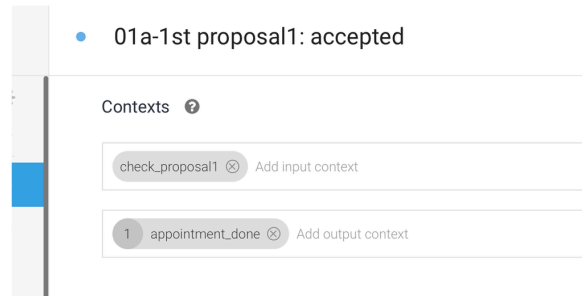
for that reason be careful to identify on which language you are when updating it.

The Intents have two forms of Context, input, and output:

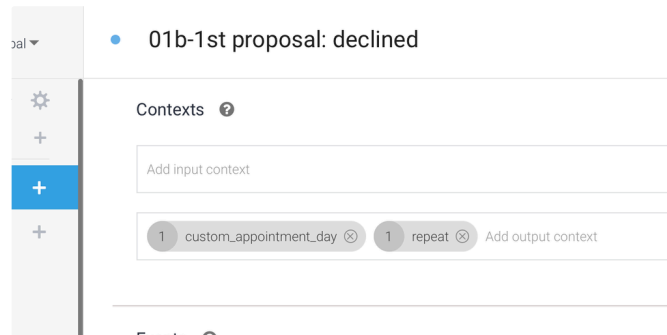


For instance the intent `00-1st-proposal` is connected to the ones that have `check_proposal1` as `input`, because the mentioned one has it as an `output`, and for that case we have at least two options:

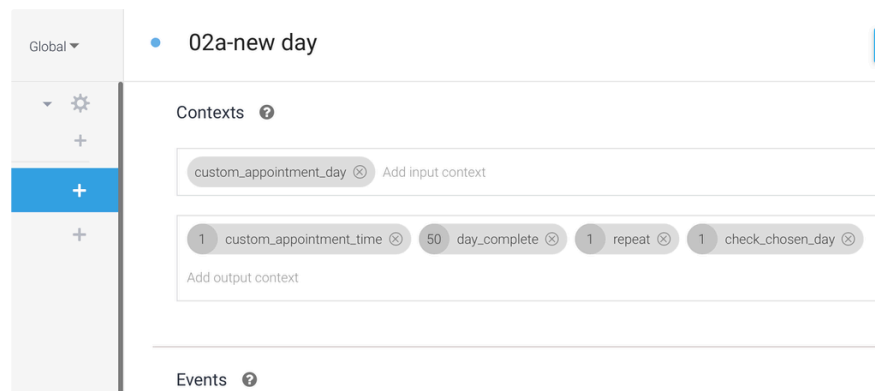
1. the user accepts the proposal



2. The user declines the proposal:



As our example is when the user declines the first proposal, then we must find an Intent which contains an `input` context with `custom_appointment_day`, in which case we have:



Training Phrases [🔗](#)

Staying with the last one, `02a-new day`, we have there `Training phrases`, where we can add possible phrases that the user could give, so the bot can recognise the `intent` (magic):

• 02a-new day

Events ?

Training phrases ?

Search training phrases

⚠ Template phrases are deprecated and will be ignored in training time. More details [here](#).

When a user says something similar to a training phrase, Dialogflow matches it to the intent. You don't have to create an Dialogflow will fill out the list with similar expressions. To extract parameter values, use [annotations](#) with available [system](#) entity types.

” Add user expression

” then i take tomorrow

” tomorrow

” the day after tomorrow

” best fit for me would be june 12th

” i would like the 27th of june

” i would prefer 14th of june

” best would be on tuesday

” on the 12th of august

” next week tuesday

” on friday

The ones with different colour are the parameters, that are defined just below the Training Phrases:

Action and parameters

replace-day-opening-times.short-answer

REQUIRED ?	PARAMETER NAME ?	ENTITY ?	VALUE	IS LIST ?
☐	date	@sys.date	\$date	☐
☐	Enter name	Enter entity	Enter value	☐

[+ New parameter](#)

For this case we expect the user to give a date, so we use the type provided by DF ([System entities reference](#) | [Dialogflow ES](#) | [Google Cloud](#)). Don't try to reinvent the wheel, as DF provides a mechanism to create custom entities, but if we can use the ones provided by the platform, use them. On the Training Phrases you can select part of the text that should be the type of Parameter you are expecting (therefore the different colour).

Parameters [🔗](#)

And just below the parameters, we have the Responses:

Responses ?

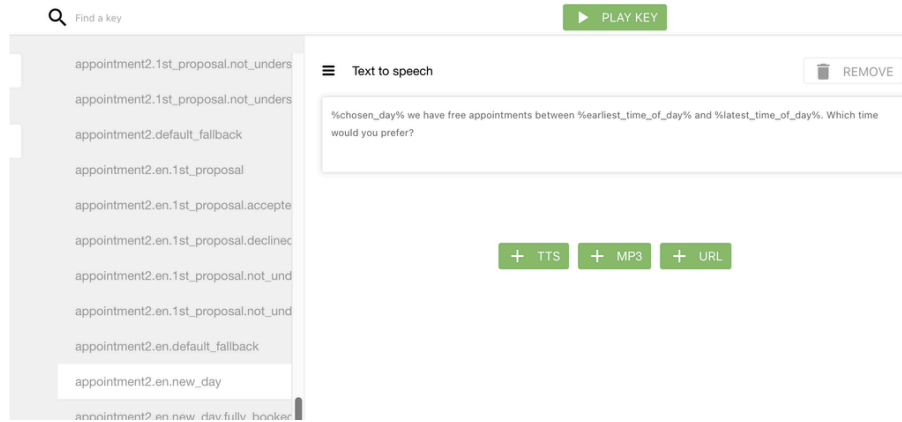
[DEFAULT](#) +

Text Response

- 1 appointment2.en.new_day
- 2 Enter a text response variant

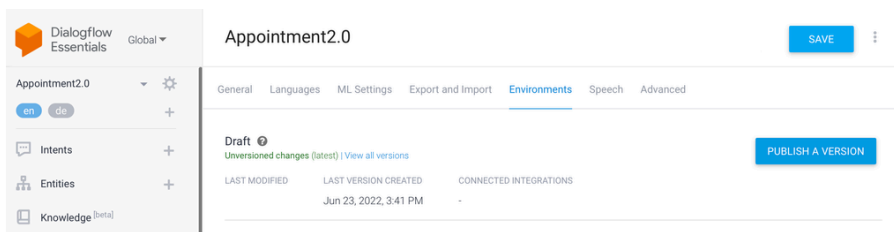
[ADD RESPONSES](#)

which are the ones that we store on Bixby, and for this particular case, `appointment2.en.new-day` is ([🔗 virtualQ](#)):

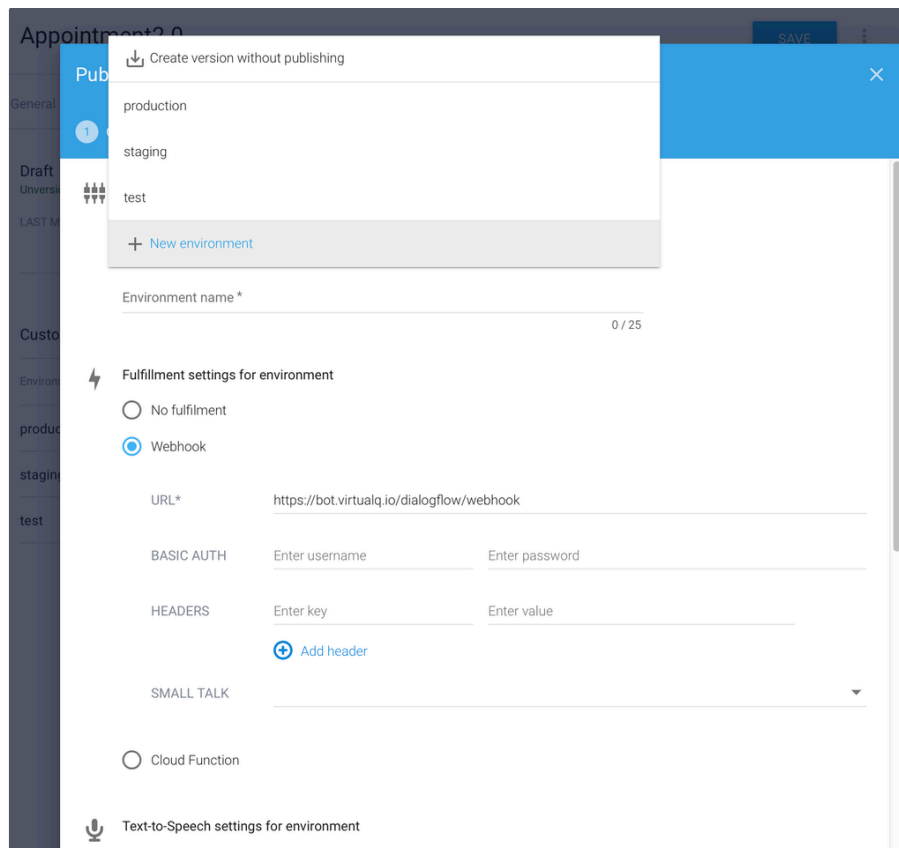


Deploying changes [🔗](#)

Whenever you make changes to the Entities or the Intents, you must deploy a new version of it. Go to Settings, Environments, Publish a new Version:

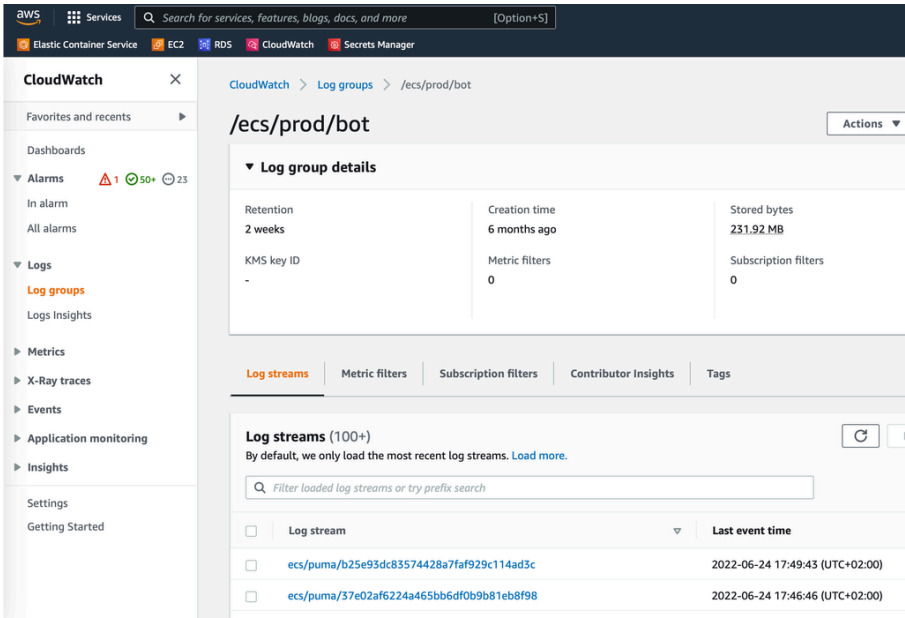


Select the environment where you want to deploy the changes (you can have different backends for each one, like staging, test, production), the url in bot is <https://bot.virtualq.io/dialogflow/webhook>, which is the endpoint that will receive the interaction from DF

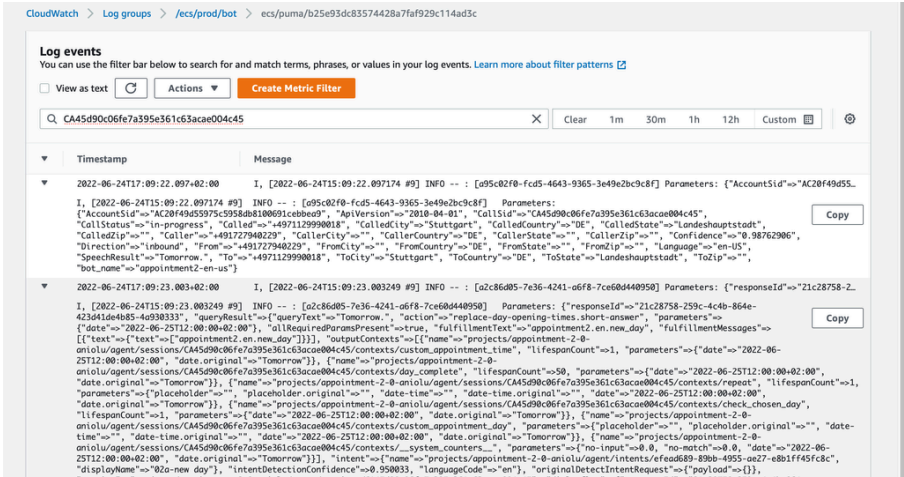


How to check the logs

When testing/debugging with a real call, you probably will want to check the logs, for that you have to go to CloudWatch



and open the `puma` stream. For filtering you will need the `CALL_SID`. Here is an example:



Part of the output is:

```
1 Parameters: {"responseId"=>"21c28758-259c-4c4b-864e-423d41de4b85-4a930333",
2 "queryResult"=>{"queryText"=>"Tomorrow.", "action"=>"replace-day-opening-times.short-answer",
3 "parameters"=>{"date"=>"2022-06-25T12:00:00+02:00"}, "allRequiredParamsPresent"=>true,
4 "fulfillmentText"=>"appointment2.en.new_day",
5 "fulfillmentMessages"=>[{"text"=>"text"=>["appointment2.en.new_day"]}]],
6 "outputContexts"=>[{"name"=>"projects/appointment-2-0-aniolu/agent/sessions/CA45d90c06fe7a395e361c63acae004c45/contexts/custom_appointment_time",
7 "lifspanCount"=>1, "parameters"=>{"date"=>"2022-06-25T12:00:00+02:00", "date.original"=>"Tomorrow"}
```

Now you can see part of what we have seen before, actions, text, etc. Make a call and check the whole output, and the whole flow.